Chapter 12. Iterators and Generators

Chương 12. Các bộ lặp và bộ tạo giá trị

Iterable objects and their associated iterators are a feature of ES6 that we've seen several times throughout this book. Arrays (including TypedArrays) are iterable, as are strings and Set and Map objects. This means that the contents of these data structures can be iterated—looped over—with the `for/of` loop, as we saw in §5.4.4:

Các đối tượng có thể được duyệt lặp đi lặp lại, cùng với các bộ lặp liên quan đến chúng, là những tính năng của ES6 mà chúng ta đã gặp nhiều lần trong cuốn sách này. Các mảng (bao gồm cả TypedArrays), các chuỗi ký tự, cũng như các đối tượng Set và Map đều thuộc loại này. Điều này có nghĩa là nội dung của các cấu trúc dữ liệu này có thể được duyệt lặp đi lặp lại, bằng cách sử dụng vòng lặp `for/of` , như chúng ta đã thấy ở mục §5.4.4.

```
let sum = 0;
for(let i of [1,2,3]) { // Loop once for each of these values
    sum += i;
}
sum   // => 6
```

Iterators can also be used with the `...` operator to expand or "spread" an iterable object into an array initializer or function invocation, as we saw in §7.1.2:

Các bộ lặp cũng có thể được sử dụng cùng với toán tử `...` để "mở rộng" một đối tượng có thể được lặp qua, thành một mảng hoặc kết quả của một hàm, như chúng ta đã thấy ở mục §7.1.2.

```
let chars = [..."abcd"]; // chars == ["a", "b", "c", "d"]
let data = [1, 2, 3, 4, 5];
Math.max(...data)        // => 5
```

Iterators can be used with destructuring assignment:

Các bộ lặp có thể được sử dụng cùng với phép gán kiểu giải cấu trúc dữ liệu:

```
let purpleHaze = Uint8Array.of(255, 0, 255, 128);
let [r, g, b, a] = purpleHaze; // a == 128
```

When you iterate a Map object, the returned values are `[key, value]` pairs, which work well with destructuring assignment in a `for/of` loop:

Khi lặp qua các phần tử trong đối tượng Map, các giá trị được trả về sẽ ở dạng các cặp `[key, value]` . Các cặp giá trị này rất phù hợp để được sử dụng trong các lệnh gán dữ liệu trong vòng lặp `for/of` .

```
let m = new Map([["one", 1], ["two", 2]]);
for(let [k,v] of m) console.log(k, v); // Logs 'one 1' and 'two 2'
```

If you want to iterate just the keys or just the values rather than the pairs, you can use the `keys()` and `values()` methods:

Nếu bạn chỉ muốn lặp lại các khóa hoặc các giá trị, chứ không phải cả cặp khóa-giá trị, bạn có thể sử dụng các phương thức `keys()` và `values()` .

```
[...m]            // => [["one", 1], ["two", 2]]: default iteration
[...m.entries()]  // => [["one", 1], ["two", 2]]: entries() method is the same
[...m.keys()]     // => ["one", "two"]: keys() method iterates just map keys
[...m.values()]   // => [1, 2]: values() method iterates just map values
```

Finally, a number of built-in functions and constructors that are commonly used with Array objects are actually written (in ES6 and later) to accept arbitrary iterators instead. The `Set()` constructor is one such API:

Cuối cùng, một số hàm và phương thức được sử dụng phổ biến khi làm việc với các đối tượng mảng thì thực chất được viết sao cho có thể chấp nhận bất kỳ đối tượng kiểu iterator nào (trong ES6 và các phiên bản sau này). Phương thức `Set()` chính là một ví dụ về API như vậy.

```
// Strings are iterable, so the two sets are the same:
new Set("abc") // => new Set(["a", "b", "c"])
```

This chapter explains how iterators work and demonstrates how to create your own data structures that are iterable. After explaining basic iterators, this chapter covers generators, a powerful new feature of ES6 that is primarily used as a particularly easy way to create iterators.

Chương này giải thích cách thức hoạt động của các bộ lặp (iterators), đồng thời hướng dẫn cách tạo ra các cấu trúc dữ liệu có thể được xử lý bằng các bộ lặp. Sau khi trình bày về các bộ lặp cơ bản, chương này sẽ đề cập đến khái niệm "generator" – một tính năng mới mạnh mẽ của ES6. Generator được sử dụng chủ yếu như một cách đơn giản để tạo ra các bộ lặp.

12.1 How Iterators Work

12.1 Cách thức hoạt động của các bộ lặp

The `for/of` loop and spread operator work seamlessly with iterable objects, but it is worth understanding what is actually happening to make the iteration work. There are three separate types that you need to understand to understand iteration in JavaScript. First, there are the *iterable* objects: these are types like Array, Set, and Map that can be iterated. Second, there is the *iterator* object itself, which performs the iteration. And third, there is the *iteration result* object that holds the result of each step of the iteration.

Toán tử vòng lặp `for/of` và toán tử phân tán hoạt động một cách trơn tru với các đối tượng có thể được lặp qua. Tuy nhiên, việc hiểu rõ những gì đang xảy ra trong quá trình lặp sẽ giúp chúng ta sử dụng chúng một cách hiệu quả hơn. Có ba thành phần chính cần được hiểu để nắm bắt cơ chế lặp trong JavaScript. Thứ nhất là các đối tượng có thể được lặp qua: đó là các kiểu dữ liệu như Array, Set, Map. Thứ hai là đối tượng iterator, chính là đối tượng thực hiện việc lặp. Thứ ba là đối tượng chứa kết quả của từng bước lặp.

An *iterable* object is any object with a special iterator method that returns an iterator object. An *iterator* is any object with a `next()` method that returns an iteration result object. And an *iteration result* object is an object with properties named `value` and `done`. To iterate an iterable object, you first call its iterator method to get an iterator object. Then, you call the `next()` method of the iterator object repeatedly until the returned value has its `done` property set to `true`. The tricky thing about this is that the iterator method of an iterable object does not have a conventional name but uses the Symbol `Symbol.iterator` as its name. So a simple `for/of` loop over an iterable object `iterable` could also be written the hard way, like this:

Một đối tượng có thể được lặp qua là bất kỳ đối tượng nào có phương thức iterator đặc biệt, phương thức này trả về một đối tượng iterator. Đối tượng iterator là bất kỳ đối tượng nào có phương thức `next()` , phương thức này trả về kết quả của việc lặp qua đối tượng đó. Đối tượng kết quả của việc lặp qua chứa các thuộc tính `value` và `done` . Để lặp qua một đối tượng có thể được lặp qua, trước tiên bạn gọi phương thức iterator của đối tượng đó để nhận được đối tượng iterator. Sau đó, bạn liên tục gọi phương thức `next()` của đối tượng iterator cho đến khi giá trị trả về có thuộc tính `done` bằng `true` . Điểm khó hiểu ở đây là phương thức iterator của đối tượng có thể được lặp qua không có tên gọi thông thường; thay vào đó, nó sử dụng ký hiệu Symbol `Symbol.iterator` làm tên gọi. Vì vậy, vòng lặp đơn giản `for/of` trên một đối tượng có thể được lặp qua `iterable` cũng có thể được viết theo cách phức tạp hơn như sau:

```
let iterable = [99];
let iterator = iterable[Symbol.iterator]();
for(let result = iterator.next(); !result.done; result = iterator.next()) {
    console.log(result.value)  // result.value == 99
}
```

The iterator object of the built-in iterable datatypes is itself iterable. (That is, it has a method named `Symbol.iterator` that just returns itself.) This is occasionally useful in code like the following when you want to iterate though a "partially used" iterator:

Đối tượng iterator của các kiểu dữ liệu có thể lặp lại có sẵn có trong ngôn ngữ lập trình cũng có khả năng được lặp lại. Nói cách khác, đối tượng này có một phương thức tên là `Symbol.iterator` , và phương thức này chỉ trả về chính đối tượng đó. Điều này đôi khi rất hữu ích trong các đoạn mã như sau, khi chúng ta muốn lặp qua một đối tượng iterator đã được sử dụng một phần.

```
let list = [1,2,3,4,5];
let iter = list[Symbol.iterator]();
let head = iter.next().value;  // head == 1
let tail = [...iter];          // tail == [2,3,4,5]
```

12.2 Implementing Iterable Objects

12.2 Triển khai các đối tượng có thể lặp lại

Iterable objects are so useful in ES6 that you should consider making your own datatypes iterable whenever they represent something that can be iterated. The Range classes shown in Examples 9-2 and 9-3 in Chapter 9 were iterable. Those classes used generator functions to make themselves iterable. We'll document generators later in this chapter, but first, we will implement the Range class one more time, making it iterable without relying on a generator.

Đối tượng có thể lặp lại rất hữu ích trong ES6. Bạn nên xem xét tạo các kiểu dữ liệu có thể lặp lại khi chúng đại diện cho một thứ gì đó có thể được lặp lại. Các lớp `Range` được hiển thị trong ví dụ 9-2 và 9-3 trong chương 9 là có thể lặp lại. Các lớp này sử dụng các hàm tạo để trở thành có khả năng lặp lại. Chúng tôi sẽ thảo luận về các hàm tạo sau trong chương này, nhưng trước tiên, chúng tôi sẽ thực hiện lại lớp `Range`, khiến nó có khả năng lặp lại mà không cần sử dụng hàm tạo.

In order to make a class iterable, you must implement a method whose name is the Symbol `Symbol.iterator`. That method must return an iterator object that has a `next()` method. And the `next()` method must return an iteration result object that has a `value` property and/or a boolean `done` property. Example 12-1 implements an iterable Range class and demonstrates how to create iterable, iterator, and iteration result objects.

Để biến một lớp thành đối tượng có thể được duyệt lặp đi lặp lại, bạn cần triển khai một phương thức có tên là `Symbol.iterator` . Phương thức này phải trả về một đối tượng iterator; đối tượng iterator này lại phải có phương thức `next()` . Ngoài ra, phương thức `next()` của đối tượng iterator phải trả về một đối tượng chứa kết quả của quá trình duyệt lặp; đối tượng này phải có thuộc tính `value` và/hoặc thuộc tính kiểu boolean là `done` . Ví dụ 12-1 minh họa cách triển khai lớp Range thành đối tượng có thể được duyệt lặp, đồng thời chỉ ra cách tạo các đối tượng iterator và kết quả của quá trình duyệt lặp.

Example 12-1. An iterable numeric Range class

Ví dụ 12-1. Lớp Range dùng để lặp qua các giá trị số

```
/*
 * A Range object represents a range of numbers {x: from <= x <= to}
 * Range defines a has() method for testing whether a given number is a member
 * of the range. Range is iterable and iterates all integers within the range.
 */
class Range {
    constructor (from, to) {
        this.from = from;
        this.to = to;
    }

    // Make a Range act like a Set of numbers
    has(x) { return typeof x === "number" && this.from <= x && x <= this.to; }

    // Return string representation of the range using set notation
    toString() { return `{ x | ${this.from} ≤ x ≤ ${this.to} }`; }

    // Make a Range iterable by returning an iterator object.
    // Note that the name of this method is a special symbol, not a string.
    [Symbol.iterator]() {
        // Each iterator instance must iterate the range independently of
        // others. So we need a state variable to track our location in the
        // iteration. We start at the first integer >= from.
        let next = Math.ceil(this.from);  // This is the next value we return
        let last = this.to;               // We won't return anything > this
        return {                          // This is the iterator object
            // This next() method is what makes this an iterator object.
            // It must return an iterator result object.
            next() {
                return (next <= last)   // If we haven't returned last value yet
                    ? { value: next++ } // return next value and increment it
                    : { done: true };   // otherwise indicate that we're done.
            },

            // As a convenience, we make the iterator itself iterable.
            [Symbol.iterator]() { return this; }
        };
    }
}

for(let x of new Range(1,10)) console.log(x); // Logs numbers 1 to 10
[...new Range(-2,2)]                          // => [-2, -1, 0, 1, 2]
```

In addition to making your classes iterable, it can be quite useful to define functions that return iterable values. Consider these iterable-based alternatives to the `map()` and `filter()` methods of JavaScript arrays:

Ngoài việc biến các lớp của bạn thành những đối tượng có thể được duyệt lặp lại từng phần, việc định nghĩa các hàm trả về giá trị có thể được duyệt cũng rất hữu ích. Hãy xem xét các giải pháp thay thế dựa trên khái niệm "đối tượng có thể được duyệt" này, thay vì sử dụng các phương thức `map()` và `filter()` trong mảng của JavaScript:

```
// Return an iterable object that iterates the result of applying f()
// to each value from the source iterable
function map(iterable, f) {
    let iterator = iterable[Symbol.iterator]();
    return {     // This object is both iterator and iterable
        [Symbol.iterator]() { return this; },
        next() {
            let v = iterator.next();
            if (v.done) {
                return v;
            } else {
                return { value: f(v.value) };
            }
        }
    };
}

// Map a range of integers to their squares and convert to an array
[...map(new Range(1,4), x => x*x)]  // => [1, 4, 9, 16]

// Return an iterable object that filters the specified iterable,
// iterating only those elements for which the predicate returns true
function filter(iterable, predicate) {
    let iterator = iterable[Symbol.iterator]();
    return { // This object is both iterator and iterable
        [Symbol.iterator]() { return this; },
        next() {
            for(;;) {
                let v = iterator.next();
                if (v.done || predicate(v.value)) {
                    return v;
                }
            }
        }
    };
}

// Filter a range so we're left with only even numbers
[...filter(new Range(1,10), x => x % 2 === 0)]  // => [2,4,6,8,10]
```

One key feature of iterable objects and iterators is that they are inherently lazy: when computation is required to compute the next value, that computation can be deferred until the value is actually needed. Suppose, for example, that you have a very long string of text that you want to tokenize into space-separated words. You could simply use the `split()` method of your string, but if you do this, then the entire string has to be processed before you can use even the first word. And you end up allocating lots of memory for the returned array and all of the strings within it. Here is a function that allows you to lazily iterate the words of a string without keeping them all in memory at once (in ES2020, this function would be much easier to implement using the iterator-returning `matchAll()` method described in §11.3.2):

Một đặc điểm quan trọng của các đối tượng có thể lặp và các trình lặp là chúng có tính "lazy" (lì hoạt động): khi cần tính toán giá trị tiếp theo, việc tính toán có thể được trì hoãn cho đến khi giá trị thực sự cần. Ví dụ, nếu bạn có một chuỗi văn bản rất dài mà bạn muốn chia thành các từ được phân tách bằng khoảng trắng, bạn có thể chỉ sử dụng phương thức `split()` của chuỗi đó. Tuy nhiên, nếu bạn làm như vậy, toàn bộ chuỗi sẽ phải được xử lý trước khi bạn có thể sử dụng từ đầu tiên. Và bạn sẽ phải phân bổ một lượng lớn bộ nhớ cho mảng trả về và tất cả các chuỗi bên trong nó. Dưới đây là một hàm cho phép bạn lặp các từ trong một chuỗi một cách "lazy" mà không cần lưu trữ tất cả chúng trong bộ nhớ cùng một lúc (trong ES2020, hàm này sẽ dễ dàng hơn nhiều để thực hiện bằng phương thức trả về trình lặp được mô tả trong §11.3.2):

```
function words(s) {
    var r = /\s+|$/g;                      // Match one or more spaces or end
    r.lastIndex = s.match(/[^ ]/).index;   // Start matching at first nonspace
    return {                               // Return an iterable iterator object
        [Symbol.iterator]() {              // This makes us iterable
            return this;
        },
        next() {                           // This makes us an iterator
            let start = r.lastIndex;       // Resume where the last match ended
            if (start < s.length) {        // If we're not done
                let match = r.exec(s);     // Match the next word boundary
                if (match) {               // If we found one, return the word
                    return { value: s.substring(start, match.index) };
                }
            }
            return { done: true };         // Otherwise, say that we're done
        }
    };
}

[...words(" abc def  ghi! ")] // => ["abc", "def", "ghi!"]
```

12.2.1 "Closing" an Iterator: The Return Method

12.2.1 "Đóng" một bộ lặp: Phương thức return

Imagine a (server-side) JavaScript variant of the `words()` iterator that, instead of taking a source string as its argument, takes the name of a file, opens the file, reads lines from it, and iterates the words from those lines. In most operating systems, programs that open files to read from them need to remember to close those files when they are done reading, so this hypothetical iterator would be sure to close the file after the `next()` method returns the last word in it.

Hãy tưởng tượng một phiên bản của bộ lặp `words()` được viết bằng JavaScript, nhưng hoạt động ở phía máy chủ. Thay vì nhận một chuỗi ký tự làm đối số, bộ lặp này sẽ nhận tên của một tệp tin. Sau đó, nó sẽ mở tệp tin đó, đọc nội dung của các dòng trong tệp, và xử lý từng từ trong các dòng đó. Trên hầu hết các hệ điều hành, các chương trình cần phải đảm bảo đóng tệp sau khi đọc xong nội dung của nó. Vì vậy, bộ lặp này chắc chắn sẽ đóng tệp ngay sau khi phương thức `next()` trả về từ cuối cùng trong tệp đó.

But iterators don't always run all the way to the end: a `for/of` loop might be terminated with a `break` or `return` or by an exception. Similarly, when an iterator is used with destructuring assignment, the `next()` method is only called enough times to obtain values for each of the specified variables. The iterator may have many more values it could return, but they will never be requested.

Nhưng các bộ lặp không phải lúc nào cũng chạy hết quá trình lặp: Vòng lặp `for/of` có thể kết thúc ở bước `break` hoặc `return` , hoặc do một ngoại lệ xảy ra. Tương tự, khi sử dụng bộ lặp trong các phép gán dữ liệu, phương thức `next()` chỉ được gọi đúng số lần cần thiết để lấy giá trị cho các biến được chỉ định. Bộ lặp có thể còn nhiều giá trị khác có thể được trả về, nhưng chúng sẽ không bao giờ được sử dụng.

If our hypothetical words-in-a-file iterator never runs all the way to the end, it still needs to close the file it opened. For this reason, iterator objects may implement a `return()` method to go along with the `next()` method. If iteration stops before `next()` has returned an iteration result with the `done` property set to `true` (most commonly because you left a `for/of` loop early via a `break` statement), then the interpreter will check to see if the iterator object has a `return()` method. If this method exists, the interpreter will invoke it with no arguments, giving the iterator the chance to close files, release memory, and otherwise clean up after itself. The `return()` method must return an iterator result object. The properties of the object are ignored, but it is an error to return a non-object value.

Nếu trình duyệt các từ trong một tệp không bao giờ đến hết, thì nó vẫn cần phải đóng tệp đã mở. Do đó, các đối tượng trình duyệt có thể triển khai phương thức `return()` để bổ sung cho phương thức `next()` . Nếu quá trình lặp dừng trước khi `next()` trả về một kết quả lặp với thuộc tính `done` được đặt thành `true` (thường là do bạn dừng một vòng lặp `for/of` bằng một câu lệnh `break`), thì trình thông dịch sẽ kiểm tra xem đối tượng trình duyệt có phương thức `return()` hay không. Nếu phương thức này tồn tại, trình thông dịch sẽ gọi nó mà không có tham số, cho trình duyệt cơ hội đóng tệp, giải phóng bộ nhớ và dọn dẹp các tài nguyên khác. Phương thức `return()` phải trả về một đối tượng kết quả trình duyệt. Các thuộc tính của đối tượng này sẽ bị bỏ qua, nhưng việc trả về một giá trị không phải là một đối tượng là một lỗi.

The `for/of` loop and the spread operator are really useful features of JavaScript, so when you are creating APIs, it is a good idea to use them when possible. But having to work with an iterable object, its iterator object, and the iterator's result objects makes the process somewhat complicated. Fortunately, generators can dramatically simplify the creation of custom iterators, as we'll see in the rest of this chapter.

Vòng lặp `for/of` và toán tử phân tán là những tính năng rất hữu ích của JavaScript. Vì vậy, khi tạo các API, bạn nên sử dụng chúng nếu có thể. Tuy nhiên, việc phải xử lý các đối tượng có thể được lặp qua, các đối tượng iterator, cũng như các kết quả của iterator tạo ra khiến quy trình trở nên phức tạp hơn một chút. May mắn thay, các hàm generator có thể giúp đơn giản hóa việc tạo các iterator tùy chỉnh một cách đáng kể, như chúng ta sẽ thấy trong phần tiếp theo của chương này.

12.3 Generators 12.3 Máy phát điện

A *generator* is a kind of iterator defined with powerful new ES6 syntax; it's particularly useful when the values to be iterated are not the elements of a data structure, but the result of a computation.

Bộ tạo giá trị (generator) là một loại công cụ dùng để lặp qua các giá trị, được định nghĩa bằng cú pháp mới mạnh mẽ của ES6. Công cụ này rất hữu ích trong trường hợp các giá trị cần được lặp qua không phải là các phần tử của một cấu trúc dữ liệu, mà là kết quả của một phép tính toán.

To create a generator, you must first define a *generator function*. A generator function is syntactically like a regular JavaScript function but is defined with the keyword `function*` rather than `function`. (Technically, this is not a new keyword, just a `*` after the keyword `function` and before the function name.) When you invoke a generator function, it does not actually execute the function body, but instead returns a generator object. This generator object is an iterator. Calling its `next()` method causes the body of the generator function to run from the start (or whatever its current position is) until it reaches a `yield` statement. `yield` is new in ES6 and is something like a `return` statement. The value of the `yield` statement becomes the value returned by the `next()` call on the iterator. An example makes this clearer:

Để tạo một trình tạo (generator), bạn cần định nghĩa một hàm tạo. Hàm tạo có cấu trúc tương tự như một hàm JavaScript thông thường, nhưng được định nghĩa bằng từ khóa `function*` thay vì `function` . (Về mặt kỹ thuật, đây không phải là một từ khóa mới, chỉ là một `*` sau từ khóa `function` và trước tên hàm.) Khi bạn gọi một hàm tạo, nó không thực sự thực hiện thân hàm, mà thay vào đó trả về một đối tượng trình tạo. Đối tượng trình tạo này là một trình duyệt. Gọi phương thức `next()` của nó sẽ thực hiện thân hàm từ đầu (hoặc vị trí hiện tại của nó) cho đến khi gặp câu lệnh `yield` . `yield` là một khái niệm mới trong ES6 và tương tự như câu lệnh `return` . Giá trị của câu `yield` sẽ trở thành giá trị trả về của cuộc gọi `next()` trên trình duyệt. Một ví dụ sẽ làm rõ điều này:

```
// A generator function that yields the set of one digit (base-10) primes.
function* oneDigitPrimes() { // Invoking this function does not run the code
    yield 2;                 // but just returns a generator object. Calling
    yield 3;                 // the next() method of that generator runs
    yield 5;                 // the code until a yield statement provides
    yield 7;                 // the return value for the next() method.
}

// When we invoke the generator function, we get a generator
let primes = oneDigitPrimes();

// A generator is an iterator object that iterates the yielded values
primes.next().value          // => 2
primes.next().value          // => 3
primes.next().value          // => 5
primes.next().value          // => 7
primes.next().done           // => true

// Generators have a Symbol.iterator method to make them iterable
primes[Symbol.iterator]()    // => primes

// We can use generators like other iterable types
[...oneDigitPrimes()]        // => [2,3,5,7]
let sum = 0;
for(let prime of oneDigitPrimes()) sum += prime;
sum                          // => 17
```

In this example, we used a `function*` statement to define a generator. Like regular functions, however, we can also define generators in expression form. Once again, we just put an asterisk after the `function` keyword:

Trong ví dụ này, chúng ta sử dụng câu lệnh `function*` để định nghĩa một "generator". Tuy nhiên, giống như các hàm thông thường, chúng ta cũng có thể định nghĩa các "generator" dưới dạng biểu thức. Để làm điều này, chúng ta chỉ cần thêm dấu sao sau từ khóa `function` .

```
const seq = function*(from,to) {
    for(let i = from; i <= to; i++) yield i;
};
[...seq(3,5)] // => [3, 4, 5]
```

In classes and object literals, we can use shorthand notation to omit the `function` keyword entirely when we define methods. To define a generator in this context, we simply use an asterisk before the method name where the `function` keyword would have been, had we used it:

Trong các lớp và các đối tượng, chúng ta có thể sử dụng cách viết tắt để bỏ qua hoàn toàn từ khóa `function` khi định nghĩa các phương thức. Để định nghĩa một hàm sinh viên (generator) trong trường hợp này, chúng ta chỉ cần sử dụng dấu sao trước tên phương thức; đó chính là vị trí mà từ khóa `function` sẽ được đặt nếu chúng ta sử dụng nó.

```
let o = {
    x: 1, y: 2, z: 3,
    // A generator that yields each of the keys of this object
    *g() {
        for(let key of Object.keys(this)) {
            yield key;
        }
    }
};
[...o.g()] // => ["x", "y", "z", "g"]
```

Note that there is no way to write a generator function using arrow function syntax.

Lưu ý rằng không thể viết hàm tạo ra các đối tượng khác nhau bằng cú pháp hàm mũi tên.

Generators often make it particularly easy to define iterable classes. We can replace the `[Symbol.iterator]()` method show in Example 12-1 with a much shorter `*[Symbol.iterator&rbrack;()` generator function that looks like this:

Các hàm tạo ra dữ liệu kiểu "iterable" thường giúp việc định nghĩa các lớp có khả năng lặp đi lặp lại trở nên dễ dàng hơn nhiều. Chúng ta có thể thay thế phương thức `[Symbol.iterator]()` trong Ví dụ 12-1 bằng một hàm tạo dữ liệu kiểu "iterable" ngắn gọn hơn, như sau:

```
*[Symbol.iterator]() {
    for(let x = Math.ceil(this.from); x <= this.to; x++) yield x;
}
```

See Example 9-3 in Chapter 9 to see this generator-based iterator function in context.

Xem Ví dụ 9-3 trong Chương 9 để hiểu rõ hơn về cách thức hoạt động của hàm lặp dựa trên bộ tạo giá trị này.

12.3.1 Generator Examples

12.3.1 Ví dụ về các bộ tạo số ngẫu nhiên

Generators are more interesting if they actually *generate* the values they yield by doing some kind of computation. Here, for example, is a generator function that yields Fibonacci numbers:

Các hàm tạo giá trị sẽ thú vị hơn nếu chúng thực sự tạo ra các giá trị đó thông qua một quy trình tính toán nào đó. Ví dụ, đây là một hàm tạo ra các số trong dãy Fibonacci:

```
function* fibonacciSequence() {
    let x = 0, y = 1;
    for(;;) {
        yield y;
        [x, y] = [y, x+y];  // Note: destructuring assignment
    }
}
```

Note that the `fibonacciSequence()` generator function here has an infinite loop and yields values forever without returning. If this generator is used with the `...` spread operator, it will loop until memory is exhausted and the program crashes. With care, it is possible to use it in a `for/of` loop, however:

Lưu ý rằng hàm tạo giá trị `fibonacciSequence()` ở đây có vòng lặp vô hạn; nó sẽ liên tục tạo ra các giá trị mà không bao giờ trả về giá trị nào cả. Nếu hàm này được sử dụng cùng toán tử phân tán `...` , chương trình sẽ tiếp tục chạy trong vòng lặp vô hạn cho đến khi bộ nhớ cạn kiệt và chương trình bị sập. Tuy nhiên, với sự cẩn trọng, vẫn có thể sử dụng hàm này trong các vòng lặp loại `for/of` .

```
// Return the nth Fibonacci number
function fibonacci(n) {
    for(let f of fibonacciSequence()) {
        if (n-- <= 0) return f;
    }
}
fibonacci(20)   // => 10946
```

This kind of infinite generator becomes more useful with a `take()` generator like this:

Loại bộ tạo số vô hạn này trở nên hữu ích hơn khi được kết hợp với bộ tạo số kiểu `take()` .

```
// Yield the first n elements of the specified iterable object
function* take(n, iterable) {
    let it = iterable[Symbol.iterator](); // Get iterator for iterable object
    while(n-- > 0) {           // Loop n times:
        let next = it.next();  // Get the next item from the iterator.
        if (next.done) return; // If there are no more values, return early
        else yield next.value; // otherwise, yield the value
    }
}

// An array of the first 5 Fibonacci numbers
[...take(5, fibonacciSequence())]  // => [1, 1, 2, 3, 5]
```

Here is another useful generator function that interleaves the elements of multiple iterable objects:

Đây là một hàm tạo dữ liệu hữu ích khác, có chức năng xen kẽ các phần tử của nhiều đối tượng có thể được duyệt lần lượt:

```
// Given an array of iterables, yield their elements in interleaved order.
function* zip(...iterables) {
    // Get an iterator for each iterable
    let iterators = iterables.map(i => i[Symbol.iterator]());
    let index = 0;
    while(iterators.length > 0) {       // While there are still some iterators
        if (index >= iterators.length) {    // If we reached the last iterator
            index = 0;                      // go back to the first one.
        }
        let item = iterators[index].next(); // Get next item from next iterator.
        if (item.done) {                    // If that iterator is done
            iterators.splice(index, 1);     // then remove it from the array.
        }
        else {                              // Otherwise,
            yield item.value;               // yield the iterated value
            index++;                        // and move on to the next iterator.
        }
    }
}

// Interleave three iterable objects
[...zip(oneDigitPrimes(),"ab",[0])]     // => [2,"a",0,3,"b",5,7]
```

12.3.2 yield* and Recursive Generators

12.3.2 Hàm yield* và các bộ tạo giá trị một cách tuần tự/đệ quy

In addition to the `zip()` generator defined in the preceding example, it might be useful to have a similar generator function that yields the elements of multiple iterable objects sequentially rather than interleaving them. We could write that generator like this:

Ngoài hàm tạo ra dữ liệu kiểu `zip()` được định nghĩa trong ví dụ trước, có thể hữu ích khi có một hàm tạo dữ liệu tương tự, nhưng hàm này sẽ lần lượt trả về các phần tử của nhiều đối tượng có thể được duyệt qua, thay vì trộn chúng lại với nhau. Chúng ta có thể viết hàm tạo dữ liệu như sau:

```
function* sequence(...iterables) {
    for(let iterable of iterables) {
        for(let item of iterable) {
            yield item;
        }
    }
}

[...sequence("abc",oneDigitPrimes())]  // => ["a","b","c",2,3,5,7]
```

This process of yielding the elements of some other iterable object is common enough in generator functions that ES6 has special syntax for it. The `yield*` keyword is like `yield` except that, rather than yielding a single value, it iterates an iterable object and yields each of the resulting values. The `sequence()` generator function that we've used can be simplified with `yield*` like this:

Quy trình truy xuất các phần tử từ một đối tượng có thể lặp lại khá phổ biến trong các hàm tạo ra dãy giá trị. Vì vậy, ES6 đã định nghĩa cú pháp riêng cho việc này. Từ khóa `yield*` tương tự như `yield` , nhưng thay vì truy xuất một giá trị duy nhất, nó sẽ lặp qua từng phần tử của đối tượng có thể lặp lại và truy xuất từng giá trị đó. Hàm tạo ra dãy giá trị `sequence()` mà chúng ta đã sử dụng có thể được đơn giản hóa bằng cách sử dụng `yield*` như sau:

```
function* sequence(...iterables) {
    for(let iterable of iterables) {
        yield* iterable;
    }
}

[...sequence("abc",oneDigitPrimes())]  // => ["a","b","c",2,3,5,7]
```

The array `forEach()` method is often an elegant way to loop over the elements of an array, so you might be tempted to write the `sequence()` function like this:

Phương thức `forEach()` thường là cách hiệu quả để lặp qua từng phần tử trong một mảng. Vì vậy, bạn có thể muốn viết hàm `sequence()` theo cách này:

```
function* sequence(...iterables) {
    iterables.forEach(iterable => yield* iterable );  // Error
}
```

This does not work, however. `yield` and `yield*` can only be used within generator functions, but the nested arrow function in this code is a regular function, not a `function*` generator function, so `yield` is not allowed.

Tuy nhiên, cách này không hoạt động được. `yield` và `yield*` chỉ có thể được sử dụng bên trong các hàm tạo ra dữ liệu kiểu "generator function". Nhưng hàm con đang mô tả trong đoạn mã này là một hàm bình thường, chứ không phải hàm tạo ra dữ liệu kiểu `function*` . Vì vậy, việc sử dụng `yield` là không được phép.

`yield*` can be used with any kind of iterable object, including iterables implemented with generators. This means that `yield*` allows us to define recursive generators, and you might use this feature to allow simple non-recursive iteration over a recursively defined tree structure, for example.

`yield*` có thể được sử dụng với mọi loại đối tượng có khả năng được lặp lại, bao gồm cả những đối tượng được tạo ra bằng cách sử dụng các hàm tạo chuỗi dữ liệu. Điều này có nghĩa là `yield*` cho phép chúng ta định nghĩa các hàm tạo chuỗi dữ liệu mang tính chất "đệ quy". Bạn có thể tận dụng tính năng này để thực hiện việc lặp qua các phần tử trong một cấu trúc dữ liệu được định nghĩa một cách đệ quy, một cách đơn giản.

12.4 Advanced Generator Features

12.4 Các tính năng nâng cao của bộ phát điện

The most common use of generator functions is to create iterators, but the fundamental feature of generators is that they allow us to pause a computation, yield intermediate results, and then resume the computation later. This means that generators have features beyond those of iterators, and we explore those features in the following sections.

Cách sử dụng phổ biến nhất của các hàm tạo bộ lặp là để tạo ra các đối tượng iterator. Tuy nhiên, đặc điểm cơ bản của các hàm tạo bộ lặp là chúng cho phép chúng ta tạm dừng quá trình tính toán, lưu lại các kết quả trung gian, rồi tiếp tục quá trình tính toán sau đó. Điều này có nghĩa là các hàm tạo bộ lặp có nhiều tính năng hơn so với các đối tượng iterator. Chúng ta sẽ tìm hiểu chi tiết về các tính năng này trong các phần tiếp theo.

12.4.1 The Return Value of a Generator Function

12.4.1 Giá trị trả về của hàm tạo ra dữ liệu (Generator Function)

The generator functions we've seen so far have not had `return` statements, or if they have, they have been used to cause an early return, not to return a value. Like any function, though, a generator function can return a value. In order to understand what happens in this case, recall how iteration works. The return value of the `next()` function is an object that has a `value` property and/or a `done` property. With typical iterators and generators, if the `value` property is defined, then the `done` property is undefined or is `false`. And if `done` is `true`, then `value` is undefined. But in the case of a generator that returns a value, the final call to `next` returns an object that has both `value` and `done` defined. The `value` property holds the return value of the generator function, and the `done` property is `true`, indicating that there are no more values to iterate. This final value is ignored by the `for/of` loop and by the spread operator, but it is available to code that manually iterates with explicit calls to `next()`:

Các hàm tạo mà chúng ta đã thấy cho đến nay không sử dụng các câu lệnh `return` , hoặc nếu chúng có sử dụng, thì chúng được sử dụng để gây ra một kết quả sớm, chứ không phải để trả về một giá trị. Tuy nhiên, giống như bất kỳ hàm nào, một hàm tạo cũng có thể trả về một giá trị. Để hiểu điều gì xảy ra trong trường hợp này, hãy nhớ cách hoạt động của vòng lặp. Giá trị trả về của hàm `next()` là một đối tượng có thuộc tính `value` và/hoặc thuộc tính `done` . Với các trình duyệt và hàm tạo thông thường, nếu thuộc tính `value` được định nghĩa, thì thuộc tính `done` sẽ không được định nghĩa hoặc sẽ là `false` . Và nếu `done` là `true` , thì `value` sẽ không được định nghĩa. Nhưng trong trường hợp một hàm tạo trả về một giá trị, thì cuộc gọi cuối cùng đến `next` sẽ trả về một đối tượng có cả `value` và `done` được định nghĩa. Thuộc tính `value` chứa giá trị trả về của hàm tạo, và thuộc tính `done` là `true` , cho biết rằng không còn giá trị nào để lặp lại. Giá trị cuối cùng này bị bỏ qua bởi vòng lặp `for/of` và toán tử phân phối dữ liệu. Tuy nhiên, giá trị này vẫn có thể được sử dụng trong các đoạn mã thực hiện việc lặp lại một cách thủ công, bằng cách gọi hàm `next()` một cách rõ ràng.

```
function *oneAndDone() {
    yield 1;
    return "done";
}

// The return value does not appear in normal iteration.
[...oneAndDone()]   // => [1]

// But it is available if you explicitly call next()
let generator = oneAndDone();
generator.next()           // => { value: 1, done: false}
generator.next()           // => { value: "done", done: true }
// If the generator is already done, the return value is not returned again
generator.next()           // => { value: undefined, done: true }
```

12.4.2 The Value of a yield Expression

12.4.2 Giá trị của biểu thức trả lời

In the preceding discussion, we've treated `yield` as a statement that takes a value but has no value of its own. In fact, however, `yield` is an expression, and it can have a value.

Trong phần thảo luận trước đó, chúng ta đã coi `yield` như một câu lệnh vốn nhận một giá trị nào đó, nhưng bản thân nó không có giá trị nào cả. Tuy nhiên, thực chất `yield` là một biểu thức, và nó hoàn toàn có thể có giá trị.

When the `next()` method of a generator is invoked, the generator function runs until it reaches a `yield` expression. The expression that follows the `yield` keyword is evaluated, and that value becomes the return value of the `next()` invocation. At this point, the generator function stops executing right in the middle of evaluating the `yield` expression. The next time the `next()` method of the generator is called, the argument passed to `next()` becomes the value of the `yield` expression that was paused. So the generator returns values to its caller with `yield`, and the caller passes values in to the generator with `next()`. The generator and caller are two separate streams of execution passing values (and control) back and forth. The following code illustrates:

Khi phương thức `next()` của một bộ tạo giá trị (generator) được gọi, hàm tạo giá trị sẽ chạy cho đến khi gặp biểu thức `yield` . Biểu thức sau từ `yield` sẽ được đánh giá, và giá trị đó sẽ trở thành giá trị trả về của lần gọi `next()` . Tại thời điểm này, hàm tạo giá trị sẽ dừng thực hiện ngay trong quá trình đánh giá biểu thức `yield` . Lần gọi tiếp theo phương thức `next()` của bộ tạo giá trị, đối số được truyền vào `next()` sẽ trở thành giá trị của biểu thức `yield` mà đã bị tạm dừng. Do đó, trình tạo sẽ trả về giá trị cho người gọi bằng `yield` , và người gọi sẽ truyền giá trị vào trình tạo bằng `next()` . Trình tạo và người gọi là hai luồng thực thi riêng biệt, trao đổi giá trị (và điều khiển) qua lại. Mã sau minh họa điều này:

```
function* smallNumbers() {
    console.log("next() invoked the first time; argument discarded");
    let y1 = yield 1;    // y1 == "b"
    console.log("next() invoked a second time with argument", y1);
    let y2 = yield 2;    // y2 == "c"
    console.log("next() invoked a third time with argument", y2);
    let y3 = yield 3;    // y3 == "d"
    console.log("next() invoked a fourth time with argument", y3);
    return 4;
}

let g = smallNumbers();
console.log("generator created; no code runs yet");
let n1 = g.next("a");   // n1.value == 1
console.log("generator yielded", n1.value);
let n2 = g.next("b");   // n2.value == 2
console.log("generator yielded", n2.value);
let n3 = g.next("c");   // n3.value == 3
console.log("generator yielded", n3.value);
let n4 = g.next("d");   // n4 == { value: 4, done: true }
console.log("generator returned", n4.value);
```

When this code runs, it produces the following output that demonstrates the back-and-forth between the two blocks of code:

Khi đoạn mã này được thực thi, nó sẽ tạo ra kết quả như sau, minh họa cho sự tương tác qua lại giữa hai khối mã này:

```
generator created; no code runs yet
next() invoked the first time; argument discarded
generator yielded 1
next() invoked a second time with argument b
generator yielded 2
next() invoked a third time with argument c
generator yielded 3
next() invoked a fourth time with argument d
generator returned 4
```

Note the asymmetry in this code. The first invocation of `next()` starts the generator, but the value passed to that invocation is not accessible to the generator.

Hãy chú ý đến sự bất đối xứng trong đoạn mã này. Lần gọi đầu tiên của `next()` sẽ khởi động bộ tạo giá trị, nhưng giá trị được truyền vào lần gọi đó lại không thể được truy cập bởi bộ tạo giá trị đó.

12.4.3 The return() and throw() Methods of a Generator

12.4.3 Các phương thức return() và throw() của một đối tượng Generator

We've seen that you can receive values yielded by or returned by a generator function. And you can pass values to a running generator by passing those values when you call the `next()` method of the generator.

Chúng ta đã thấy rằng chúng ta có thể nhận các giá trị được tạo ra hoặc trả về bởi một hàm tạo giá trị. Ngoài ra, chúng ta cũng có thể truyền các giá trị vào hàm tạo giá trị đó bằng cách truyền chúng khi gọi phương thức `next()` của hàm tạo giá trị đó.

In addition to providing input to a generator with `next()`, you can also alter the flow of control inside the generator by calling its `return()` and `throw()` methods. As the names suggest, calling these methods on a generator causes it to return a value or throw an exception as if the next statement in the generator was a `return` or `throw`.

Ngoài việc cung cấp dữ liệu vào bộ tạo giá trị (`next()`), bạn còn có thể thay đổi luồng xử lý bên trong bộ tạo giá trị này bằng cách gọi các phương thức `return()` và `throw()` . Như tên gọi của chúng, việc gọi các phương thức này sẽ khiến bộ tạo giá trị trả về một giá trị nhất định, hoặc ném ra ngoại lệ; điều này tương tự như việc thực thi câu lệnh tiếp theo trong bộ tạo giá trị đó là một phương thức `return` hoặc `throw` .

Recall from earlier in the chapter that, if an iterator defines a `return()` method and iteration stops early, then the interpreter automatically calls the `return()` method to give the iterator a chance to close files or do other cleanup. In the case of generators, you can't define a custom `return()` method to handle cleanup, but you can structure the generator code to use a `try/finally` statement that ensures the necessary cleanup is done (in the `finally` block) when the generator returns. By forcing the generator to return, the generator's built-in `return()` method ensures that the cleanup code is run when the generator will no longer be used.

Như đã đề cập trước đó, nếu một trình duyệt định nghĩa một phương thức `return()` , và quá trình lặp dừng sớm, thì trình thông dịch sẽ tự động gọi phương thức `return()` để cho trình duyệt cơ hội đóng file hoặc thực hiện các thao tác làm sạch khác. Trong trường hợp của các trình tạo (generators), bạn không thể định nghĩa một phương thức tùy chỉnh `return()` để xử lý việc làm sạch, nhưng bạn có thể cấu trúc mã của trình tạo để sử dụng một câu lệnh `try/finally` , đảm bảo rằng các thao tác làm sạch cần thiết (trong khối `finally`) sẽ được thực hiện khi trình tạo trả về. Bằng cách buộc trình tạo trả về, phương thức tích hợp của trình tạo (`return()`) đảm bảo rằng mã làm sạch sẽ được thực hiện khi trình tạo không còn được sử dụng.

Just as the `next()` method of a generator allows us to pass arbitrary values into a running generator, the `throw()` method of a generator gives us a way to send arbitrary signals (in the form of exceptions) into a generator. Calling the `throw()` method always causes an exception inside the generator. But if the generator function is written with appropriate exception-handling code, the exception need not be fatal but can instead be a means of altering the behavior of the generator. Imagine, for example, a counter generator that yields an ever-increasing sequence of integers. This could be written so that an exception sent with `throw()` would reset the counter to zero.

Tương tự như phương pháp `next()` cho phép chúng ta truyền các giá trị tùy ý vào một trình tạo, phương pháp `throw()` cho một trình tạo cho phép chúng ta gửi các tín hiệu tùy ý (dưới dạng ngoại lệ) vào trình tạo. Gọi phương pháp `throw()` luôn gây ra một ngoại lệ bên trong trình tạo. Tuy nhiên, nếu hàm trình tạo được viết với mã xử lý ngoại lệ thích hợp, ngoại lệ không nhất thiết phải gây ra lỗi, mà có thể là một cách để thay đổi hành vi của trình tạo. Ví dụ, hãy tưởng tượng một trình tạo đếm số, tạo ra một chuỗi các số nguyên luôn tăng. Trình này có thể được viết sao cho khi một ngoại lệ được gửi qua `throw()` , nó sẽ reset lại đếm số về 0.

When a generator uses `yield*` to yield values from some other iterable object, then a call to the `next()` method of the generator causes a call to the `next()` method of the iterable object. The same is true of the `return()` and `throw()` methods. If a generator uses `yield*` on an iterable object that has these methods defined, then calling `return()` or `throw()` on the generator causes the iterator's `return()` or `throw()` method to be called in turn. All iterators *must* have a `next()` method. Iterators that need to clean up after incomplete iteration *should* define a `return()` method. And any iterator *may* define a `throw()` method, though I don't know of any practical reason to do so.

Khi một bộ tạo (generator) sử dụng ` yield* ` để lấy giá trị từ một đối tượng có thể lặp lại, thì việc gọi phương thức ` next() ` của bộ tạo sẽ gọi phương thức ` next() ` của đối tượng có thể lặp lại. Điều này cũng đúng với các phương thức ` return() ` và ` throw() `. Nếu một bộ tạo sử dụng ` yield* ` trên một đối tượng có thể lặp lại và có các phương thức này được định nghĩa, thì việc gọi ` return() ` hoặc ` throw() ` trên bộ tạo sẽ gọi phương thức ` return() ` hoặc ` throw() ` của trình duyệt (iterator). Tất cả các trình duyệt phải có phương thức ` next() `. Các trình duyệt cần phải dọn dẹp sau khi hoàn thành nên nên định nghĩa phương thức ` return() `. Và bất kỳ trình duyệt nào cũng có thể định nghĩa phương thức ` throw() `, mặc dù tôi không biết lý do thực tế để làm điều này.

12.4.4 A Final Note About Generators

12.4.4 Một vài lưu ý cuối cùng về các bộ tạo xung điện

Generators are a very powerful generalized control structure. They give us the ability to pause a computation with `yield` and restart it again at some arbitrary later time with an arbitrary input value. It is possible to use generators to create a kind of cooperative threading system within single-threaded JavaScript code. And it is possible to use generators to mask asynchronous parts of your program so that your code appears sequential and synchronous, even though some of your function calls are actually asynchronous and depend on events from the network.

Các bộ tạo giá trị (generators) là những cấu trúc điều khiển rất mạnh mẽ. Chúng cho phép ta tạm dừng một quá trình tính toán bằng lệnh `yield` , sau đó khởi động lại quá trình đó vào một thời điểm bất kỳ trong tương lai, với các giá trị đầu vào tùy ý. Các bộ tạo giá trị còn có thể được sử dụng để tạo ra một hệ thống xử lý nhiều luồng trong JavaScript một luồng. Ngoài ra, chúng còn giúp che giấu những phần không đồng bộ trong chương trình, khiến cho mã nguồn trông giống như đang được thực hiện một cách tuần tự và đồng bộ, mặc dù một số lệnh gọi hàm thực chất lại được thực hiện một cách bất đồng bộ và phụ thuộc vào các sự kiện từ mạng.

Trying to do these things with generators leads to code that is mind-bendingly hard to understand or to explain. It has been done, however, and the only really practical use case has been for managing asynchronous code. JavaScript now has `async` and `await` keywords (see Chapter 13) for this very purpose, however, and there is no longer any reason to abuse generators in this way.

Việc cố gắng sử dụng các công cụ tạo mã nguồn tự động để thực hiện những việc này sẽ dẫn đến kết quả là đoạn mã trở nên rất khó hiểu và khó giải thích. Tuy nhiên, cách làm này vẫn từng được áp dụng. Trường hợp sử dụng hữu ích duy nhất của việc này chính là để quản lý các đoạn mã được thực thi một cách bất đồng bộ. Hiện nay, JavaScript đã có các từ khóa `async` và `await` dành riêng cho mục đích này (xem Chương 13). Vì vậy, không còn lý do gì để lạm dụng các công cụ tạo mã nguồn tự động nữa.

12.5 Summary 12.5 Tóm tắt

In this chapter, you have learned:

Trong chương này, bạn đã học được những điều sau:

- The `for/of` loop and the `...` spread operator work with iterable objects.

  Vòng lặp `for/of` và toán tử phân tán `...` hoạt động hiệu quả với các đối tượng có thể được duyệt lần lượt.
- An object is iterable if it has a method with the symbolic name `[Symbol.iterator]` that returns an iterator object.

  Một đối tượng được coi là có thể được duyệt lần lượt nếu nó có một phương thức mang tên ký hiệu `[Symbol.iterator]` , và phương thức này trả về một đối tượng dùng để duyệt các phần tử của đối tượng đó.
- An iterator object has a `next()` method that returns an iteration result object.

  Đối tượng iterator có phương thức `next()` , phương thức này trả về một đối tượng chứa kết quả của quá trình lặp.
- An iteration result object has a `value` property that holds the next iterated value, if there is one. If the iteration has completed, then the result object must have a `done` property set to `true`.

  Đối tượng kết quả của mỗi lần lặp chứa thuộc tính `value` , giá trị của thuộc tính này chính là giá trị tiếp theo trong lần lặp tiếp theo, nếu có. Nếu quá trình lặp đã hoàn tất, thì thuộc tính `done` của đối tượng kết quả phải được đặt thành giá trị `true` .
- You can implement your own iterable objects by defining a `[Symbol.iterator]()` method that returns an object with a `next()` method that returns iteration result objects. You can also implement functions that accept iterator arguments and return iterator values.

  Bạn có thể tự tạo ra các đối tượng có khả năng lặp lại bằng cách định nghĩa một phương thức `[Symbol.iterator]()` , phương thức này sẽ trả về một đối tượng khác. Đối tượng được trả về này lại có phương thức `next()` , phương thức này sẽ trả về các kết quả của quá trình lặp. Bạn cũng có thể tạo ra các hàm nhận các đối tượng kiểu iterator làm đối số, và trả về các giá trị tương ứng từ các đối tượng iterator đó.
- Generator functions (functions defined with `function*` instead of `function`) are another way to define iterators.

  Các hàm tạo ra các bộ lặp (những hàm được định nghĩa bằng `function*` thay vì `function`) là một cách khác để định nghĩa các bộ lặp.
- When you invoke a generator function, the body of the function does not run right away; instead, the return value is an iterable iterator object. Each time the `next()` method of the iterator is called, another chunk of the generator function runs.

  Khi bạn gọi một hàm sinh viên (generator function), thân hàm đó sẽ không được thực thi ngay lập tức. Thay vào đó, giá trị trả về sẽ là một đối tượng iterator có khả năng được lặp đi lặp lại. Mỗi lần phương thức `next()` của đối tượng iterator được gọi, một phần nội dung của hàm sinh viên sẽ được thực thi.
- Generator functions can use the `yield` operator to specify the values that are returned by the iterator. Each call to `next()` causes the generator function to run up to the next `yield` expression. The value of that `yield` expression then becomes the value returned by the iterator. When there are no more `yield` expressions, then the generator function returns, and the iteration is complete.

  Các hàm tạo ra dữ liệu có thể sử dụng toán tử `yield` để chỉ định các giá trị mà bộ lặp sẽ trả về. Mỗi lần gọi toán tử `next()` , hàm tạo dữ liệu sẽ thực thi cho đến khi gặp biểu thức `yield` tiếp theo. Giá trị của biểu thức `yield` đó sẽ trở thành giá trị được trả về bởi bộ lặp. Khi không còn biểu thức nào nữa, hàm tạo dữ liệu sẽ kết thúc việc thực thi của mình, và quá trình lặp cũng hoàn tất.